

CS 2110 - Spring 2026

Project 5: Dynamic Memory

Andy Garcha, Bryce Teto, Nishi Agrawal, William Sugden, Seha Park,
James P Davis, Aadit Bansal, Rohit Rao, Daniel Huang,
Jason Tran, Cole Saunders, Tanay Kapoor, Chris Goggins

Due: 4/19/2026 @ 11:59 PM

Version: 1.1

NOTE: This project was previously due 4/17/2026, a Friday. This due date has been moved to 4/19/2026, a Sunday, in exchange for having **NO LATE SUBMISSIONS**.

Contents

1	General	3
1.1	Introduction	3
1.2	What functions can I use?	3
2	Data Structures	3
2.1	Linked Lists	3
2.1.1	Initialization	4
2.1.2	Add to Back	4
2.1.3	Remove from Back	4
2.1.4	Contains	5
2.1.5	Destroying	5
3	Structs and Global Variables	5
3.1	Enums and Structs	6
3.1.1	Item	6
3.1.2	ItemNode	6
3.1.3	TableOrder	6
3.1.4	Menu	6
3.2	Global Variables	6
3.2.1	Orders	6
3.2.2	Menu	7
4	Instructions	8
4.1	General instructions	8

4.2	Functions to Implement	8
4.2.1	Menu Operations	8
4.2.2	TableOrder Operations	8
4.2.3	Restaurant Management Operations	9
4.2.4	Destroying	9
4.3	Common Mistakes	9
4.3.1	Forgetting to check for malloc failures	9
4.3.2	Incorrectly handling realloc failures	10
4.3.3	Incorrect size when dynamically allocating strings	10
4.3.4	Incorrectly handling malloc failures in chains of mallocs	10
4.3.5	Forgetting to free dynamically allocated fields when freeing struct instances	11
4.3.6	Incorrectly freeing linked list nodes	11
5	Building & Testing	12
5.1	Helpful Info	12
5.1.1	Man Pages	12
5.1.2	Debugging with GDB and printf	12
5.2	Checking Your Solution	12
5.2.1	Makefiles	12
5.2.2	Valgrind	13
5.2.3	Autograder	13
5.3	Important Notes	14
6	Deliverables	14
7	Rules and Regulations	14
7.1	General Rules	14
7.2	Submission Conventions	14
7.3	Submission Guidelines	14
7.4	Coding Guidelines C/C++ code	15
7.5	Syllabus Excerpt on Academic Misconduct	15

1 General

1.1 Introduction

Welcome to project 5 restaurant lovers! You’ve recently opened a restaurant and have found that you need a better solution for managing orders, menus, and starving customers, so you decided to use your knowledge of dynamic memory allocation to create a better system for your staff. In this project, you’ll be mimicking the behavior of a restaurant order platform to allow you to keep track of items and their menu status, add table orders & costs, and even shut down your restaurant when the going gets tough.

We have provided templates for functions in `restaurant.h` that you will be implementing in `restaurant.c`. This document will contain more detailed instructions as well as some additional information that will be useful as you work on the project.

Please read through the entire document before starting. The *Data Structures* section will explain how to implement the data structures used in this project as well as several of the operations that you will need to implement for those data structures. The *Common Mistakes* section will cover several common mistakes that are easy to make when working with dynamic memory allocation in C. Be sure to read both of those sections thoroughly, as they may be very useful when you are implementing your functions.

Start early and if you get stuck, there’s always Piazza and office hours. Bon appétit!

1.2 What functions can I use?

1. You **are** allowed to make your own helper functions in `restaurant.c` if you so desire.
2. You can use the following library functions in your project: `printf`, `strlen`, `strcpy/strncpy`, `strncpy/strncmp`, `malloc`, `calloc`, `realloc`.
3. Do **NOT** modify or add your own include statements. Including a new header file is **NOT** allowed and the autograder will get mad at you. This includes **ANY** standard library headers not already provided.

2 Data Structures

For this project, you will need to know how linked lists work as parts of the project utilize these two data structures. This section will give an overview of how linked lists work as well as how to implement some of their operations that you will need to use in this project.

2.1 Linked Lists

A linked list is a type of dynamically sized list. In a linked list, we use pointers to chain together nodes that contain our data. Each node contains the data for that element and a pointer to the next node. Each node is dynamically allocated individually, so our elements are not necessarily contiguous in memory. The first node of the linked list is the “head,” and we will keep a pointer to this node so that we can traverse through the chain from the beginning to reach any node in the list.

We need to define a struct for our linked list nodes—we will call it `ListNode`. Each node will hold some data and a pointer to the next node, so we will have two fields: `data` and `next`. For demonstration here, we will use an `Element` struct as the data we are storing, so `data` will have type `Element`. The `next` field is a pointer to the next node, so it will have type `ListNode *`.

```
typedef struct ListNode {  
    Element data;
```

```

    ListNode *next;
} ListNode;

```

2.1.1 Initialization

To create a new linked list, we have to initialize our `head` pointer. We don't have any elements in our linked list yet, so the head pointer will be `NULL`. Since we mark the end of our linked list by having the final node point to `NULL` in our implementation, we don't need a variable to keep track of the size. So, the initialization will simply look like this:

```

ListNode *head = NULL;

```

2.1.2 Add to Back

To add an element to the back of our linked list, we have to dynamically allocate a new `ListNode` for that element, follow the chain of pointers to reach the final node in the list, and update the `next` field of that final node to point to our new node. This looks something like this:

```

ListNode *new_node_ptr = (ListNode *) malloc(sizeof(ListNode));
if (new_node_ptr == NULL) {
    return FAILURE;
}
new_node_ptr->data = {
    // Assign struct fields
};
new_node_ptr->next = NULL;

ListNode *curr = head;
while (curr->next != NULL) {
    curr = curr->next;
}
curr->next = new_node_ptr;

```

As always, we check for `malloc` failure when we allocate our new `ListNode` struct instance. Because the final node in the chain points to `NULL`, we can traverse to the final node using a while loop that will break once we reach a node whose `next` field is `NULL`. However, there is one scenario that this code fails to consider: If our list is currently empty, then `curr` will be initialized to `NULL` (since `head` is `NULL`), and trying to access `curr->next` in our while loop will cause a null pointer dereference because if the list is empty, then there is no final node to traverse to. So, we have to check for this case up front:

```

// malloc new_node_ptr and set its fields as we did before
if (head == NULL) {
    head = new_node_ptr;
} else {
    // iterate to back and add
}

```

2.1.3 Remove from Back

To remove a node from the back of our linked list, we have to traverse through every node of the list until we reach the tail (remember how we checked if a node was the tail in `Add to Back`). Once we get there, we will free that node and ensure that the node directly before it points to null as it's next. This would look something like this:

```

ListNode *curr = head;

if (curr->next == NULL) { // one node in list
    free(curr);
    head = NULL;
    return;
}

while (curr->next->next != NULL) {
    curr = curr->next;
}
free(curr->next);
curr->next = NULL;

```

Additionally, note that if our `ListNode` struct has any dynamically allocated fields, we must free those before we free the pointer to the node instance itself.

2.1.4 Contains

Say we have some `target` value and we would like to check if our linked list contains an element whose `value` field matches `target`. To do this, we need to iterate through every node in the list, checking whether the `value` field of each element equals `target`. We will return `SUCCESS` if we find such an element, otherwise we will return `FAILURE`. For example:

```

ListNode *curr = head;
while (curr != NULL) {
    if (curr->data.value == target) {
        return SUCCESS;
    }
    curr = curr->next;
}
return FAILURE;

```

2.1.5 Destroying

To destroy our linked list, we have to iterate through the chain of list nodes, freeing each one. Because we can't access the `next` field after freeing `node`, we can use a temporary copy of the `next` pointer. We can do this:

```

ListNode *curr = head;

while (curr != NULL) {
    ListNode *next = curr->next;
    free(curr);
    curr = next;
}

```

3 Structs and Global Variables

For this project, we have defined various structs and global variables in `restaurant.h` for you to use. It will be useful to understand all of these before you start writing your code.

3.1 Enums and Structs

3.1.1 Item

```
typedef struct Item {
    char[MAX_ITEM_LEN] name;
    int order_count;
    int cook_time; // amount of time it takes to cook
    int cost;
} Item;
```

The `Item` struct is used for representing information about a single item. Users will keep a `Menu` struct of `ItemNode` structs (that uses `Item`) as a Singly Linked List.

3.1.2 ItemNode

```
typedef struct ItemNode {
    struct Item* item;
    struct ItemNode* next;
} ItemNode;
```

The `ItemNode` struct represents one `Item`. The `item` field is a pointer to the `Item`, and the `next` field is a pointer to the next `ItemNode` in the `LinkedList`.

3.1.3 TableOrder

```
typedef struct TableOrder {
    struct ItemNode *head;
    int order_time;
    int table_num;
    int guests_at_table;
} TableOrder;
```

The `TableOrder` struct represents one table's order. It's got a Linked List of items in the menu, a time it was ordered, the table number, and the amount of guests at the table.

3.1.4 Menu

```
typedef struct Menu {
    struct ItemNode *head;
    int last_updated;
} Menu;
```

The `Menu` struct represents all of the items currently offered in a menu. The `head` field points to the start of a Linked List.

3.2 Global Variables

3.2.1 Orders

```
TableOrder orders[MAX_ORDER_COUNT]
```

This will act as the global TableOrder array, which contains all TableOrders from all tables. Keep track of the number of TableOrders in the variable `num_table_orders`.

3.2.2 Menu

Menu menu

This will act as the global Restaurant menu. There is only one instance of the menu at all times! Make sure to consider how to allocate memory when adding and deleting items from the menu.

4 Instructions

4.1 General instructions

You have been given 2 C files: `restaurant.h` and `restaurant.c`. All of the functions you need to implement are in `restaurant.c` and will be explained briefly below. However, you should look at the `restaurant.h` file first as it provides definitions for the structs that you will use throughout the project as well as declarations of the functions that you are required to implement.

You can also find a list of a few common mistakes with dynamic memory allocation in C in the *Common Mistakes* section. Be sure to read this section before starting on your function implementations as it may save you from some difficult debugging.

You can use any function included in `stdlib.h`, `string.h`, and `stdio.h` as they are included in `restaurant.h`. **Including any other library is forbidden.**

You should not be leaking memory in any of the functions. For any functions using `malloc`, if `malloc` returns `null`, return `FAILURE` and ensure that no memory is leaked. The autograder catches memory leaks with `Valgrind`. See the *Valgrind* section for details.

4.2 Functions to Implement

For each of these functions, handle `NULL` or invalid arguments by returning `FAILURE` or `NULL`, as per the instructions.

4.2.1 Menu Operations

- `int addToMenu(char name[MAX_ITEM_LEN], int cook_time, int cost);`
Adds a new `Item` with name `name`, cooking time `cook_time`, and cost `cost` to the menu linked list. The function must allocate memory for both an `Item` and an `ItemNode`. If an item with the same name already exists on the menu, the item should not be added. Returns `FAILURE` upon `malloc` failure, invalid inputs, or duplicate item names, and `SUCCESS` otherwise.
- `int removeFromMenu(Item *item);`
Removes the `Item` pointed to by `item` from the menu. Ensure comparisons are done with pointer & data field entries. If the given pointer does not exist in the menu, return `FAILURE`. Otherwise, remove the item from the linked list and return `SUCCESS`.

4.2.2 TableOrder Operations

- `int addTableOrder(ItemNode *head, int table_num, int guests_at_table, int order_time);`
Adds a new `TableOrder` to the global `orders` array using the provided linked list `head` of ordered items. The table number, guest count, and order time should be stored in the new order. For each item in the list, increment that item's `order_count`. Returns `FAILURE` if inputs are invalid or if the maximum number of orders has been reached, and `SUCCESS` otherwise.
- `int calculateTableCost(TableOrder order);`
Calculates and returns the total cost of all items in given `TableOrder`. The cost is determined by summing the `cost` field of each `Item` referenced in the order. This function should not free any memory.
- `int orderOutOfDate(TableOrder order);`
Iterates through the items in `order` and checks whether each item still exists in the menu since the menu dynamically changes. Returns the number of items in the order that are no longer present on the menu.

4.2.3 Restaurant Management Operations

- `int keepChefFromQuitting();`
The chef doesn't like his job and if it becomes too much, he will quit! As Restaurant Manager, your goal is to prevent this. The method removes all items from the `menu` whose cooking time exceeds `ACCEPTABLE_COOKING_TIME`. Afterward, increase the cost of the first five items in the menu by \$2 each. Returns `SUCCESS` if the operation completes successfully and `FAILURE` if any invalid conditions occur.
- `int freeDessert(TableOrder *order);`
You hired a new server. They are still getting used to such a bustling restaurant environment so they accidentally lose orders sometimes. However, you don't want to lose customers. If your server makes a mistake for that `TableOrder`, add a free apple pie to their order. If an item named "apple pie" does not already exist in the menu, it should be added with a cost of \$0. The dessert should then be added to the corresponding order.
- `TableOrder findOldestOrder();`
Searches the global `orders` array and returns the `TableOrder` with the earliest `order_time`. If no orders exist, return `INCOMPLETE` or the appropriate invalid result.
- `ItemNode *rushHourSearch();`
Iterates over the `menu` and constructs a new linked list containing the five items with the shortest cooking times. The resulting list should be ordered from shortest to longest cook time. All `ItemNodes` in the returned list must be newly allocated. If fewer than five items exist in the menu, return `NULL`.

4.2.4 Destroying

- `int freeTableOrder(int table_num);`
Frees the `TableOrder` stored in orders at index `table_num`. An `Item` should only be freed if no other references to it exist in any table order or in the menu. After freeing the order's nodes, remove the order from the array while ensuring that there are no empty gaps in the array when you delete an order.
Hint: We recommend using helper functions to make that functionality simpler. There are a couple of ways to do this, swapping, moving everything after the freed spot up to a spot, and so on.
- `int shutdownRestaurant();`
Frees all dynamically allocated memory associated with the restaurant system, including all table orders and menu nodes. Make sure no memory leaks remain before the program exits. Returns `SUCCESS` if all memory is freed correctly and `FAILURE` otherwise.

4.3 Common Mistakes

There are several traps that are easy to fall into while using dynamic memory allocation in C. Make sure to keep these in mind while you implement the functions in this project.

4.3.1 Forgetting to check for malloc failures

If `malloc` fails to find a block of our desired size, it will return `NULL`. This means that every time we use `malloc`, we must check that the pointer it returns is not `NULL`. If we don't check the return value of `malloc` and continue as normal, then we will encounter a null pointer dereference later when we try to access the block that we just tried to allocate. In this project, we handle `malloc` failures by returning `FAILURE`, which is a macro that has been defined for you. For example:

```
MyStruct *my_ptr = malloc(size);
if (my_ptr == NULL) {
```

```

    return FAILURE;
}

```

4.3.2 Incorrectly handling realloc failures

If `realloc` fails to find a new block of our desired size, it will return `NULL` and leave the original block allocated in its current location. For example, consider this code that attempts to reallocate `my_ptr` to a new block:

```

my_ptr = realloc(my_ptr, new_size);
if (my_ptr == NULL) {
    return FAILURE;
}

```

It looks like this code is checking for `realloc` failure, but it is actually doing so incorrectly. With this code, we will lose our original `my_ptr` pointer in the event of `realloc` failure, as `my_ptr` will get reassigned to `NULL`. This will create a memory leak, as our original block will remain allocated when `realloc` fails, but we no longer have any way to access that block. To avoid losing our original pointer and properly check for `realloc` failure, we have to use a temporary pointer to hold the return value of `realloc`:

```

MyStruct *temp_ptr = realloc(my_ptr, new_size);
if (temp_ptr == NULL) {
    return FAILURE;
}
my_ptr = temp_ptr;

```

4.3.3 Incorrect size when dynamically allocating strings

Remember that every string must end with a null terminator (`'\0'` character) to indicate that this is the end of the string. The `strlen` function from `string.h` returns the length of the string ignoring the null terminator, so you have to remember to add 1 for the total size when allocating space for a string. For example, if we want to allocate a block for a string `word`, we would do the following:

```

char *word_ptr = malloc(strlen(word) + 1); // CORRECT

```

rather than this:

```

char *word_ptr = malloc(strlen(word)); // INCORRECT

```

4.3.4 Incorrectly handling malloc failures in chains of mallocs

If we have a chain of mallocs (for example, when allocating a struct instance and its fields), we need to remember to free everything that we allocated earlier in the chain in reverse order if we get a `malloc` failure. For example, say we have the following struct:

```

typedef struct Person {
    char *first_name;
    char *last_name;
} Person;

```

If we want to dynamically allocate a new instance of `Person` given strings `my_first_name` and `my_last_name` for the two fields, we have to do the following:

```

Person *person = malloc(sizeof(Person));
if (person == NULL) {
    return FAILURE;
}
person->first_name = malloc(strlen(my_first_name) + 1);
if (person->first_name == NULL) {
    free(person);          // IMPORTANT PART
    return FAILURE;
}
person->last_name = malloc(strlen(my_last_name) + 1);
if (person->last_name == NULL) {
    free(person->first_name); // IMPORTANT PART
    free(person);          // IMPORTANT PART
    return FAILURE;
}

```

If we do not free everything that we allocated earlier in the chain upon `malloc` failure, we will create a memory leak. Furthermore, if we free things in the wrong order, we will create a use-after-free error—in this example, this is why we must free `person->first_name` before `person` within the last if statement. An easy way to avoid this is by simply freeing everything in the opposite order as how they were allocated.

4.3.5 Forgetting to free dynamically allocated fields when freeing struct instances

If we have a dynamically allocated struct instance that we would like to free, we have to remember to free any dynamically allocated fields on that struct instance before freeing the struct instance itself. For example, using the same `Person` struct from above, we would free a `Person` struct instance called `person` by doing the following:

```

free(person->first_name);
free(person->last_name);
free(person);

```

If we only did `free(person)`, we would cause a memory leak, as the `first_name` and `last_name` fields would never get freed.

4.3.6 Incorrectly freeing linked list nodes

When freeing all nodes of a linked list, we might naively try to iterate through the list and free each node like this:

```

// BAD CODE
ListNode *curr = head;
while (curr != NULL) {
    free(curr);
    curr = curr->next;
}

```

However, this will result in undefined behavior—after we free `curr`, we no longer have access to the fields of that struct instance, so trying to access `curr->next` on the following line is a use-after-free error. To fix this, we have to save a pointer to the next node before freeing the current node, which we can do like this:

```

// FIXED CODE
ListNode *curr = head;

```

```

ListNode *next;

while (curr != NULL) {
    next = curr->next;
    free(curr);
    curr = next;
}

```

5 Building & Testing

5.1 Helpful Info

5.1.1 Man Pages

The `man` command in Linux provides “an interface to the on-line reference manuals.” This is a great utility for any C and Linux developer for finding out more information about the available functions and libraries. In order to use this, you just need to pass in the function name to this command within a Linux (in our case Docker) terminal.

For instance, entering the following command will print the corresponding man page for the `strlen` function:

```
$ man strlen
```

Additionally, the man pages are accessible online at: <http://man.he.net>

NOTE: You can ignore the subsections after the “RETURN VALUE” (such as ATTRIBUTES, etc) for this homework, however, pay close attention to function descriptions.

5.1.2 Debugging with GDB and printf

We highly recommend getting used to “`printf` debugging” in C. By sprinkling some `printf` statements throughout your code, you can track variable values, program flow, and state changes at various points, making it easier to identify and fix bugs.

Moreover, If you run into a problem when working on your homework, you can use the debugging tool, GDB, to debug your code! Former TA Adam Suskin made a series of tutorial videos which you can find [here](#). GDB is an essential tool for any C program, so we definitely recommend learning it now.

When running GDB, if you get to a point where user input is needed, you can supply it just like you normally would. When an error happens, you can get a Java-esque stacktrace using the `backtrace` (`bt`) command which allows you to pinpoint where the error is coming from. For more info on basic GDB commands, search up “GDB Cheat Sheet”.

5.2 Checking Your Solution

5.2.1 Makefiles

Make is a common build tool for abstracting the complexity of working with compilers directly. Makefiles let you define a set of desired targets (files you want to compile), their prerequisites (files which are needed to compile the target), and sets of directives (commands such as `gcc`, `gdb`, etc.) to build those targets. In all of our C assignments (and also in production level C projects), a Makefile is used to compile C programs with a long list of compiler flags that control things like how much to optimize the code, whether to create debugging information for `gdb`, and what errors we want to show. We have already provided you a Makefile

for this homework, but we highly recommend that you take a look at this file and understand the `gcc` commands and flags used to understand how to compile C programs. If you're interested, you can also find more information regarding Makefiles [here](#).

Since your program is connected to an autograder with multiple files that need to be compiled using particular settings, it's a little difficult to compile it by hand. The Makefile allows us to simply type the command `make` followed by a target such as `tests`, `run-tests`, or `run-gdb` to compile and run your code.

Please **DO NOT** modify your makefile as it may break your local autograder.

5.2.2 Valgrind

Valgrind is an open-source memory debugging tool. You can use it to check your code for various memory management errors, such as memory leaks, use-after-free errors, and so on. To test for memory management errors in your code, you can use `make run-valgrind` from within the Docker container as shown in the next section.

5.2.3 Autograder

The autograder must be run with the provided Docker container. To enter the Docker container, `cd` into the project directory and enter:

```
# macOS or Linux
./cs2110docker-c.sh

# Windows
.\cs2110docker-c.bat
```

If you are on macOS or Linux, you may get an error such as

```
zsh: permission denied: ./cs2110docker-c.sh
```

If so, make sure to enable execute permissions on the script: `chmod +x ./cs2110docker-c.sh`. You should only need to do this once.

Once you are in the Docker container, you can run the autograder:

```
# Clean your directory (remove any made files)
$ make clean

# Build and run all tests
$ make

# Build and run one test
$ make TEST=test_add_table_order::valid_order

# Build and run all tests in a single file
$ make TEST=test_add_table_order::*

# Build and run all tests in GDB
# run-gdb takes the same parameters above
$ make run-gdb

# Build and run with valgrind to check for memory leaks
$ make run-valgrind
```

5.3 Important Notes

1. Only the `restaurant.c` file will be graded on Gradescope.
2. All non-compiling homework will receive a zero (with all the flags specified in the Makefile/Syllabus).
3. **NOTE: DO NOT MODIFY THE HEADER FILES.**

Since you are not turning them in, any changes to the `.h` files will not be reflected when running the Gradescope autograder.

We reserve the right to update the autograder and the test case weights on Gradescope or the local checker as we see fit when grading your solution.

6 Deliverables

Turn in `restaurant.c` to Gradescope by the due date.

Please do not wait until the last minute to run/test your project, history has proved that last minute turn-ins will result in long queue times for grading on Gradescope. You have been warned!!!

7 Rules and Regulations

7.1 General Rules

1. Although you may ask TAs for clarification, you are ultimately responsible for what you submit.
2. Please read the assignment in its entirety before asking questions.
3. Please start assignments early, and ask for help early. Do not email us a few hours before the assignment is due with questions.
4. If you find any problems with the assignment, it would be greatly appreciated if you reported them to the author (which can be found at the top of the assignment). Announcements will be posted if the assignment changes.

7.2 Submission Conventions

1. In order to submit your assignment, submit the files individually to the Gradescope assignment.
2. Do not submit links to files. The autograder does not understand it, and we will not manually grade assignments submitted this way as it is easy to change the files after the submission period ends.

7.3 Submission Guidelines

1. You are responsible for turning in assignments on time. This includes allowing for unforeseen circumstances. If you have an emergency let us know **IN ADVANCE** of the due time supplying documentation (i.e. note from the dean, doctor's note, etc). Extensions will only be granted to those who contact us in advance of the deadline and no extensions will be made after the due date.
2. You are also responsible for ensuring that what you turned in is what you meant to turn in. After submitting you should be sure to download your submission into a brand new folder and test if it works. No excuses if you submit the wrong files, what you turn in is what we grade. In addition, your assignment must be turned in via Gradescope. Under no circumstances whatsoever we will accept any

email submission of an assignment. Note: if you were granted an extension you will still turn in the assignment over Gradescope.

3. Projects turned in late receive partial credit within the first 48 hours, as defined in the syllabus. Between 0 and 24 hours late, you can receive a maximum score of 70%. Between 24 and 48 hours late, you can receive a maximum score of 50%. We will not accept projects turned in over 48 hours late.
4. You alone are responsible for submitting your project before the assignment is due; neither Canvas/Gradescope, nor your flaky internet are to blame if you are unable to submit because you banked on your computer working up until the deadline.

7.4 Coding Guidelines C/C++ code

1. You must turn in ALL files specified in the “Deliverables” section of the assignment instructions. We reserve the right to impose a penalty on submissions that do not follow the given submission directions.
2. You must provide a Makefile that compiles and links your code by default. If you are given a Makefile with the project, we expect your code to compile with `make`.
3. Your code must compile with gcc on Ubuntu 22.04 LTS (aka compile using the CS 2110 Docker container). If your code does not compile, you will receive a 0 for the assignment.
4. You will be penalized if your code produces warnings when compiled with the given Makefile, or the following flags if no Makefile is provided: `gcc -Wall -Wextra -Wstrict-prototypes -pedantic -O2`
5. Code should be well commented and use a clean, consistent (readable) style (i.e., proper indenting, etc.). We reserve the right to impose style requirements, and deduct for non-conforming solutions. This is not the obfuscated C code competition (<http://ioccc.org>).

7.5 Syllabus Excerpt on Academic Misconduct

The goal of all assignments in CS 2110 is for you to learn. Learning requires thought and hard work. Copying answers thus prevents learning. More importantly, it gives an unfair advantage over students who do the work and follow the rules.

1. **As a Georgia Tech student, you have read and agreed to the Georgia Tech Academic Honor Code.** The Academic Honor Code defines Academic Misconduct as “*any act that does or could improperly distort Student grades or other Student academic records.*”
2. You must submit an assignment or project as your own work. **No collaboration on answers is permitted. Absolutely no code or answers may be copied from others. Such copying is Academic Misconduct.**
3. Using code from GitHub, via Googling, from Stack Overflow, etc., is Academic Misconduct (Honor Code: Academic Misconduct includes “*submission of material that is wholly or substantially identical to that created or published by another person or persons*”).
4. Publishing your assignments on public repositories, github, etc, that is accessible to other students is unauthorized collaboration and thus Academic Misconduct.
5. Suspected Academic Misconduct will be reported to the Division of Student Life Office of Student Integrity. It will be prosecuted to the full extent of Institute policies.
6. Students suspected of Academic Misconduct are informed **at the end of the semester**. Suspects receive an *Incomplete* final grade until the issue is resolved.
7. We use accepted forensic techniques to determine whether there is copying of a coding assignment.

8. Submitting an assignment with code or text from an AI assistant (e.g., ChatGPT) is academic misconduct.
9. **If you are not sure about any aspect of this policy, please ask your lecturer.**

Using AI Assistants

Anything you did not write in your assignment will be treated as an academic misconduct case. If you are unsure where the line is between collaborating with AI and copying AI, we recommend the following heuristics:

Heuristic 1: Never hit “Copy” within your conversation with an AI assistant. You can copy your own work into your own conversation, but do not copy anything from the conversation back into your assignment. Instead, use your interaction with the AI assistant as a learning experience, then let your assignment reflect your improved understanding.

Heuristic 2: Do not have your assignment and the AI agent open at the same time. Similar to the above, use your conversation with the AI as a learning experience, then close the interaction down, open your assignment, and let your assignment reflect your revised knowledge. This heuristic includes avoiding using AI directly integrated into your composition environment: just as you should not let a classmate write content or code directly into your submission, so also you should avoid using tools that directly add content to your submission.

Deviating from these heuristics does not automatically qualify as academic misconduct; however, following these heuristics essentially guarantees your collaboration will not cross the line into misconduct.